code_blocks

Pixels — Code Blocks Reference

> Scratch-style copy-paste building blocks for OpenCV C++.
> Each block is one logical unit. Pick the blocks you need, stack them in order, done.

Table of Contents

1. Setup & Compile
2. Program Setup
3. Image Input / Output
4. Video Input
5. Display
6. Color Space Conversion
7. Pixel Access & Modification
8. Image Geometry
9. Drawing on Images
10. Affine Transformations
11. Kernels (copy-ready matrices)
12. Filtering & Blurring
13. Masking & Bitwise Operations
14. Morphology
15. Contours
16. Blob Detection Helpers
17. Image Segmentation (GrabCut)

---

0. Setup & Compile

Step 1 — Check if OpenCV is installed

```
pkg-config --modversion opencv4
```

If a version number prints (e.g. `4.5.4`), skip to Step 3.

Step 2 — Install OpenCV (if not found)

```
sudo apt update && sudo apt install -y libopencv-dev
```

Step 3 — Compile and run your solution

```
g++ solution.cpp -o solution $(pkg-config --cflags --libs ope
ncv4) && ./solution
```

---

1. Program Setup

Includes (use this at the top of every file)

```
#include<bits/stdc++.h>
#include<opencv2/opencv.hpp>
using namespace std;
```

Minimal includes (lighter, topic-specific)

```
#include<iostream>
#include<opencv2/highgui.hpp>
#include<opencv2/imgcodecs.hpp>
#include<opencv2/imgproc.hpp>
using namespace std;
```

Main function shell

```
int main() {

    // --- your blocks go here ---

    return 0;
}
```

---

2. Image Input / Output

Load a colour image

```
cv::Mat img = cv::imread("./assets/picture.jpeg");

if (img.empty()) {
    cout << "Failed to load image" << endl;
    return -1;
}
```

Load a grayscale image

```
cv::Mat img = cv::imread("./assets/picture.jpeg", cv::IMREAD_
GRAYSCALE);

if (img.empty()) {
    cout << "Failed to load image" << endl;
    return -1;
}
```

Load from command-line argument

```
// Usage: ./output <image_path>
int main(int argc, char **argv) {
    if (argc != 2) {
        cout << "usage: ./output <Image_Path>\n";
        return -1;
    }
    cv::Mat img = cv::imread(argv[1], cv::IMREAD_GRAYSCALE);
    if (!img.data) {
        cout << "Could not open or find the image\n";
        return 0;
    }
}
```

Save (write) an image to disk

```
cv::imwrite("./assets/output.png", img);
```

Get image dimensions

```
int width    = img.size().width;   // number of columns
int height   = img.size().height;  // number of rows
int channels = img.channels();     // 1 = grayscale, 3 = BGR
```

---

3. Video Input

Open webcam (live feed)

```
cv::VideoCapture video(0);  // 0 = default camera

if (!video.isOpened()) {
    cout << "Failed to open camera" << endl;
```

```
    return -1;
}
```

Open a video file

```
cv::VideoCapture video("./assets/video.mp4");

if (!video.isOpened()) {
    cout << "Failed to open video" << endl;
    return -1;
}
```

Read frames in a loop

```
while (video.isOpened()) {
    cv::Mat frame;
    video.read(frame);  // read one frame

    // --- process frame here ---

    cv::imshow("Output", frame);

    if (cv::waitKey(10) == 'x') {  // press 'x' to quit
        cv::destroyAllWindows();
        video.release();
        break;
    }
}
```

Wait for keypress to pause / select ROI

```
// Inside video loop — press 'q' to pause and select region
if (cv::waitKey(10) == 'q') {
    cv::Rect bbox = cv::selectROI("Image", frame, false, fals
```

```
e);
    // bbox.x, bbox.y, bbox.width, bbox.height now hold the s
election
    break;
}
```

---

4. Display

Show image in window (waits for any key)

```
cv::imshow("Window Name", img);
cv::waitKey(0);
```

Show with resizable window

```
cv::namedWindow("Window Name", cv::WINDOW_NORMAL);
cv::resizeWindow("Window Name", 640, 480);
cv::imshow("Window Name", img);
cv::waitKey(0);
```

Close all windows

```
cv::destroyAllWindows();
```

Show multiple images side by side (separate windows)

```
cv::imshow("Original", img);
cv::imshow("Processed", output);
cv::waitKey(0);
cv::destroyAllWindows();
```

---

5. Color Space Conversion

BGR → Grayscale

```
cv::Mat gray;
cv::cvtColor(img, gray, cv::COLOR_BGR2GRAY);
```

BGR → HSV

```
cv::Mat hsv;
cv::cvtColor(img, hsv, cv::COLOR_BGR2HSV);
```

BGR → RGB

```
cv::Mat rgb;
cv::cvtColor(img, rgb, cv::COLOR_BGR2RGB);
```

Grayscale → BGR (to draw colour on grayscale)

```
cv::Mat bgr;
cv::cvtColor(gray, bgr, cv::COLOR_GRAY2BGR);
```

---

6. Pixel Access & Modification

Read a single pixel value (colour image)

```
// Pixel at row i, column j
cv::Vec3b pixel = img.at<cv::Vec3b>(i, j);
int blue  = pixel[0];
int green = pixel[1];
int red   = pixel[2];
```

Write a single pixel value (colour image)

```
img.at<cv::Vec3b>(i, j)[0] = 255;  // Blue channel
img.at<cv::Vec3b>(i, j)[1] = 0;    // Green channel
img.at<cv::Vec3b>(i, j)[2] = 0;    // Red channel
```

Read a single pixel value (grayscale)

```
uchar pixel = img.at<uchar>(i, j);
```

Write a single pixel value (grayscale)

```
img.at<uchar>(i, j) = 128;
```

Loop over every pixel (colour)

```
for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        cv::Vec3b pixel = img.at<cv::Vec3b>(i, j);
        // read:  pixel[0]=B, pixel[1]=G, pixel[2]=R
        // write: img.at<cv::Vec3b>(i, j)[0] = value;
    }
}
```

Loop over every pixel (grayscale)

```
for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        uchar val = img.at<uchar>(i, j);
        // modify: img.at<uchar>(i, j) = new_val;
    }
}
```

Black out alternate rows

```
for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (i % 2 == 0) {
            img.at<cv::Vec3b>(i, j)[0] = 0;
            img.at<cv::Vec3b>(i, j)[1] = 0;
            img.at<cv::Vec3b>(i, j)[2] = 0;
        }
    }
}
```

Max out one channel on alternate rows (e.g. boost blue)

```
for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (i % 2 == 0) {
            img.at<cv::Vec3b>(i, j)[0] = 255;  // change inde
x for G(1) or R(2)
        }
    }
}
```

---

7. Image Geometry

Create a blank black image

```
// (height, width, type, fill colour)
cv::Mat blank(480, 640, CV_8UC3, cv::Scalar(0, 0, 0));
```

Create a blank image same size as another

```
cv::Mat output{img.size(), img.type()};
```

Create zeros/ones matrix

```
cv::Mat zeros = cv::Mat::zeros(rows, cols, CV_8UC1);
cv::Mat ones  = cv::Mat::ones(rows, cols, CV_8UC1);
```

Resize to fixed dimensions

```
cv::Mat resized;
cv::resize(img, resized, cv::Size(width, height));
// e.g. cv::Size(640, 480)
```

Resize by scale factor

```
cv::Mat resized;
cv::resize(img, resized, cv::Size(), 0.5, 0.5, cv::INTER_CUBI
C);
// Replace 0.5 with desired scale. INTER_CUBIC = high quality
```

Resize to half

```
cv::Mat resized;
cv::resize(img, resized, cv::Size(img.cols/2, img.rows/2));
```

Crop (ROI using range slicing)

```
// img(Range(start_row, end_row), Range(start_col, end_col))
cv::Mat cropped = img(cv::Range(0, img.rows/2), cv::Range(0,
img.cols/2));
```

Crop using Rect

```
// Rect(x, y, width, height) — x=col, y=row
```

```
cv::Mat roi = img(cv::Rect(0, 0, 200, 200)).clone();
```

Clone (deep copy)

```
cv::Mat copy = img.clone();
```

---

8. Drawing on Images

Create canvas and draw a horizontal line

```
cv::Mat canvas(480, 720, CV_8UC3, cv::Scalar(0, 0, 0));  // b
lack canvas

// Horizontal line at row 200, from col 100 to 620
for (int j = 100; j <= 620; j++) {
    canvas.at<cv::Vec3b>(200, j)[0] = 255;
    canvas.at<cv::Vec3b>(200, j)[1] = 255;
    canvas.at<cv::Vec3b>(200, j)[2] = 255;
}
```

Draw a vertical line

```
// Vertical line at col 300, from row 80 to 400
for (int i = 80; i <= 400; i++) {
    canvas.at<cv::Vec3b>(i, 300)[0] = 255;
    canvas.at<cv::Vec3b>(i, 300)[1] = 255;
    canvas.at<cv::Vec3b>(i, 300)[2] = 255;
}
```

Draw a 45° diagonal line

```
// Diagonal from (100,100) to (300,300)
for (int i = 100; i <= 300; i++) {
```

```
    for (int j = 100; j <= 300; j++) {
        if (i == j) {
            canvas.at<cv::Vec3b>(i, j) = {255, 255, 255};
        }
    }
}
```

---

9. Affine Transformations

Translation

```
float tx = float(img.cols) / 4;   // shift right by 1/4 width
float ty = float(img.rows) / 4;   // shift down by 1/4 height

float warp_values[] = {1.0, 0.0, tx, 0.0, 1.0, ty};
cv::Mat translation_matrix = cv::Mat(2, 3, CV_32F, warp_value
s);

cv::Mat translated;
cv::warpAffine(img, translated, translation_matrix, img.size
());
```

Rotation around centre

```
double angle = 90;   // degrees — positive = counter-clockwis
e
double scale = 1.0;

cv::Point2f centre(img.cols / 2.0, img.rows / 2.0);
cv::Mat rotation_matrix = cv::getRotationMatrix2D(centre, ang
le, scale);

cv::Mat rotated;
```

```
cv::warpAffine(img, rotated, rotation_matrix, cv::Size(img.co
ls, img.rows));
```

Rotation as a reusable function

```
cv::Mat rotate(cv::Mat src, double angle) {
    cv::Mat dst;
    cv::Point2f pt(src.cols / 2., src.rows / 2.);
    cv::Mat r = cv::getRotationMatrix2D(pt, angle, 1.0);
    cv::warpAffine(src, dst, r, cv::Size(src.cols, src.row
s));
    return dst;
}
// Usage: cv::Mat rotated = rotate(img, 90);
```

---

10. Kernels (copy-ready matrices)

Sobel X — vertical edge detection

```
cv::Mat sobel_x = (cv::Mat_<double>(3, 3) <<
    -1., 0., 1.,
    -2., 0., 2.,
    -1., 0., 1.
);
```

Sobel Y — horizontal edge detection

```
cv::Mat sobel_y = (cv::Mat_<double>(3, 3) <<
    -1., -2., -1.,
     0.,  0.,  0.,
     1.,  2.,  1.
);
```

Laplacian — all-direction edge detection

```
cv::Mat laplacian = (cv::Mat_<double>(3, 3) <<
    -1., -1., -1.,
    -1.,  8., -1.,
    -1., -1., -1.
);
```

Box Blur — uniform average

```
cv::Mat box_blur = (cv::Mat_<double>(3, 3) <<
    1./9, 1./9, 1./9,
    1./9, 1./9, 1./9,
    1./9, 1./9, 1./9
);
```

Gaussian Blur 3×3

```
cv::Mat gaussian_3 = (cv::Mat_<double>(3, 3) <<
    1./16, 2./16, 1./16,
    2./16, 4./16, 2./16,
    1./16, 2./16, 1./16
);
```

Gaussian Blur 5×5

```
cv::Mat gaussian_5 = (cv::Mat_<double>(5, 5) <<
    1./273,  4./273,  7./273,  4./273, 1./273,
    4./273, 16./273, 26./273, 16./273, 4./273,
    7./273, 26./273, 41./273, 26./273, 7./273,
    4./273, 16./273, 26./273, 16./273, 4./273,
    1./273,  4./273,  7./273,  4./273, 1./273
);
```

Sharpening

```
cv::Mat sharpen = (cv::Mat_<double>(3, 3) <<
     0., -1.,  0.,
    -1.,  5., -1.,
     0., -1.,  0.
);
```

Identity (no change)

```
cv::Mat identity = (cv::Mat_<double>(3, 3) <<
    0., 0., 0.,
    0., 1., 0.,
    0., 0., 0.
);
```

---

11. Filtering & Blurring

Apply any kernel using filter2D (OpenCV built-in)

```
cv::Mat output;
// -1 = output same depth as input, Point(-1,-1) = kernel cen
tre, 0.0 = bias
cv::filter2D(img, output, -1, kernel, cv::Point(-1, -1), 0.0,
cv::BORDER_REPLICATE);
```

Apply kernel with offset (bias)

```
cv::Mat output;
cv::filter2D(img, output, -1, sobel, cv::Point(-1, -1), 5.0,
cv::BORDER_REPLICATE);
// 5.0 = brightness offset added after convolution
```

Gaussian Blur (OpenCV built-in)

```
cv::Mat blurred;
cv::GaussianBlur(img, blurred, cv::Size(5, 5), 0);
// cv::Size(5,5) = kernel size (must be odd). 0 = auto sigma
```

Median Blur (good for noise removal before masking)

```
cv::Mat blur;
cv::medianBlur(masked, blur, 5);  // 5 = kernel size (must be
odd)
```

Add padding/border to image

```
cv::Mat image_pad;
// BORDER_REPLICATE copies edge pixels into border
cv::copyMakeBorder(img, image_pad, 1, 1, 1, 1, cv::BORDER_REP
LICATE, cv::Scalar(0));
// Arguments: src, dst, top, bottom, left, right, border_typ
e, value
```

---

12. Masking & Bitwise Operations

Threshold (binary mask from grayscale)

```
cv::Mat mask;
// Pixels > 180 → 255 (white). Pixels <= 180 → 0 (black)
cv::threshold(gray_img, mask, 180, 255, cv::THRESH_BINARY);
```

Invert a mask

```
cv::Mat mask_inv;
```

```
cv::bitwise_not(mask, mask_inv);
```

HSV colour range mask (inRange)

```
// HSV format bounds: (hue, saturation, value)
cv::Scalar lower(15, 30, 150);   // min hue, sat, val
cv::Scalar upper(36, 255, 255);  // max hue, sat, val

cv::Mat mask;
cv::inRange(hsv, lower, upper, mask);
```

Common HSV ranges

```
// RED:    lower(0, 100, 0)    upper(10, 255, 255)
// YELLOW: lower(15, 30, 150)  upper(36, 255, 255)
// GREEN:  lower(36, 52, 80)   upper(86, 255, 255)
// BLUE:   lower(94, 52, 80)   upper(126, 255, 255)
```

Dynamic HSV bounds from median (used in blob detection)

```
// h, s, v are median values from getMedianPixelValues()
cv::Scalar lower(h - 5, max(0, s - 50), max(0, v - 50));
cv::Scalar upper(h + 5, min(s + 50, 255), min(v + 50, 255));

cv::Mat masked;
cv::inRange(hsv, lower, upper, masked);
```

Combine two masks (OR)

```
cv::Mat combined_mask;
cv::bitwise_or(mask1, mask2, combined_mask);
```

Apply mask to image (AND)

```
cv::Mat result;
cv::bitwise_and(img, img, result, mask);
// Keeps pixels where mask=255, blacks out where mask=0
```

Apply mask with separate foreground/background

```
// Black out background in img1 using mask
cv::Mat img1_bg;
cv::bitwise_and(roi, roi, img1_bg, mask);

// Extract foreground from img2 using inverse mask
cv::Mat img2_fg;
cv::bitwise_and(img2, img2, img2_fg, mask_inv);

// Combine: background + foreground
cv::Mat result;
cv::add(img1_bg, img2_fg, result);
```

Colour-based background removal (pixel loop)

```
cv::Mat masking(cv::Mat image) {
    cv::Vec3b grey  = cv::Vec3b(230, 230, 230);
    cv::Vec3b white = cv::Vec3b(255, 255, 255);
    cv::Mat output{image.size(), image.type()};

    for (int i = 0; i < image.rows; i++) {
        for (int j = 0; j < image.cols; j++) {
            cv::Vec3b x = image.at<cv::Vec3b>(i, j);
            if (grey != x && white != x) {
                output.at<cv::Vec3b>(i, j) = x;  // keep non-
background pixels
            }
        }
    }
```

```
    return output;
}
// Usage: cv::Mat clean = masking(image);
```

Blend two images

```
cv::Mat blended;
// output = alpha * img1 + beta * img2 + gamma
cv::addWeighted(img1, 0.5, img2, 0.5, 0, blended);
// Adjust 0.5/0.5 weights and 0 gamma as needed
```

---

13. Morphology

Erosion (shrinks white regions)

```
// Requires: erosion() from morphology.cpp
cv::Mat output{source.size(), source.type()};
int kernel_size = 3;  // must be odd: 3, 5, 7...
output = erosion(source, output, kernel_size);
```

Dilation (grows white regions)

```
cv::Mat output{source.size(), source.type()};
int kernel_size = 3;
output = dilation(source, output, kernel_size);
```

Opening = Erosion → Dilation (removes small noise)

```
cv::Mat temp{source.size(), source.type()};
cv::Mat output{source.size(), source.type()};
```

```
temp   = erosion(source, temp, 3);   // erode first
output = dilation(temp, output, 3);  // then dilate
```

Closing = Dilation → Erosion (fills holes)

```
cv::Mat temp{source.size(), source.type()};
cv::Mat output{source.size(), source.type()};

temp   = dilation(source, temp, 3);  // dilate first
output = erosion(temp, output, 3);   // then erode
```

Morphological Gradient = Dilation – Erosion (extracts edges)

```
cv::Mat dilated{source.size(), source.type()};
cv::Mat eroded{source.size(), source.type()};
cv::Mat gradient{source.size(), source.type()};

dilated  = dilation(source, dilated, 3);
eroded   = erosion(source, eroded, 3);
gradient = difference(dilated, eroded, gradient);
```

OpenCV built-in morphology (simpler)

```
cv::Mat kernel = cv::getStructuringElement(cv::MORPH_RECT, c
v::Size(3, 3));

cv::Mat eroded, dilated;
cv::erode(img, eroded, kernel);
cv::dilate(img, dilated, kernel);

cv::Mat opened, closed;
cv::morphologyEx(img, opened, cv::MORPH_OPEN,    kernel);
```

```
cv::morphologyEx(img, closed, cv::MORPH_CLOSE,   kernel);
cv::morphologyEx(img, gradient, cv::MORPH_GRADIENT, kernel);
```

---

14. Contours

Find contours (from binary/edge image)

```
vector<vector<cv::Point>> contours;
vector<cv::Vec4i> hierarchy;

cv::findContours(binary_img, contours, hierarchy,
                 cv::RETR_TREE,           // retrieval mode
                 cv::CHAIN_APPROX_SIMPLE  // approximation me
thod
);
// binary_img must be grayscale binary (from threshold/Canny/
inRange mask)
```

Draw ALL contours on image

```
for (size_t i = 0; i < contours.size(); i++) {
    cv::drawContours(img, contours, i, cv::Scalar(0, 255, 0),
2);
    //                                  colour (BGR)
thickness
}
```

Find and draw LARGEST contour only

```
int largest_idx = 0;
double max_area = 0;
for (int i = 0; i < (int)contours.size(); i++) {
    double area = cv::contourArea(contours[i]);
```

```
    if (area > max_area) {
        max_area = area;
        largest_idx = i;
    }
}
cv::drawContours(img, contours, largest_idx, cv::Scalar(0, 25
5, 255), 2);
```

Full contour pipeline (Grayscale → Canny → Find → Draw)

```
// Step 1: Grayscale
cv::Mat gray;
cv::cvtColor(img, gray, cv::COLOR_BGR2GRAY);

// Step 2: Edge detection
cv::Mat edges;
cv::Canny(gray, edges, 170, 250);  // lower threshold, upper
threshold

// Step 3: Find contours
vector<vector<cv::Point>> contours;
vector<cv::Vec4i> hierarchy;
cv::findContours(edges, contours, hierarchy, cv::RETR_EXTERNA
L, cv::CHAIN_APPROX_NONE);

// Step 4: Draw contours
for (size_t i = 0; i < contours.size(); i++) {
    cv::drawContours(img, contours, i, cv::Scalar(0, 255, 0),
2);
}
cv::imshow("Contours", img);
cv::waitKey(0);
```

---

15. Blob Detection Helpers

Median of a vector

```
double median(vector<double> vec) {
    int size = vec.size();
    if (size == 0) return 0;
    sort(vec.begin(), vec.end());
    if (size % 2 == 0)
        return (vec[size/2 - 1] + vec[size/2]) / 2.0;
    else
        return vec[size/2];
}
```

Get median HSV from a region

```
tuple<double, double, double> getMedianPixelValues(cv::Mat im
g) {
    vector<double> hue_pixels, saturation_pixels, value_pixel
s;

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            cv::Vec3b pixel = img.at<cv::Vec3b>(i, j);
            hue_pixels.push_back(pixel[0]);
            saturation_pixels.push_back(pixel[1]);
            value_pixels.push_back(pixel[2]);
        }
    }
    return make_tuple(median(hue_pixels), median(saturation_p
ixels), median(value_pixels));
}
```

Select ROI and extract median HSV (interactive calibration)

```
int h = 0, s = 0, v = 0;
```

```
while (video.isOpened()) {
    cv::Mat frame;
    video.read(frame);
    cv::imshow("Image", frame);

    if (cv::waitKey(10) == 'q') {
        cv::Rect bbox = cv::selectROI("Image", frame, false,
false);

        cv::Mat hsv;
        cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);
        cv::Mat obj_img = hsv(cv::Rect(bbox.x, bbox.y, bbox.w
idth, bbox.height));

        tuple<double, double, double> medians = getMedianPixe
lValues(obj_img);
        h = get<0>(medians);
        s = get<1>(medians);
        v = get<2>(medians);
        break;
    }
}
```

Full blob detection processing loop

```
while (video.isOpened()) {
    cv::Mat frame;
    video.read(frame);

    // 1. Convert to HSV
    cv::Mat hsv;
    cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);

    // 2. Create dynamic mask from calibrated h, s, v
    cv::Scalar lower(h - 5, max(0, s - 50), max(0, v - 50));
```

```
    cv::Scalar upper(h + 5, min(s + 50, 255), min(v + 50, 25
5));
    cv::Mat masked;
    cv::inRange(hsv, lower, upper, masked);

    // 3. Median blur to clean mask
    cv::Mat blur;
    cv::medianBlur(masked, blur, 5);

    // 4. Apply mask to original frame
    cv::Mat blob_mask;
    cv::bitwise_and(frame, frame, blob_mask, blur);
    cv::imshow("blob_mask", blob_mask);

    // 5. Find contours on blur mask
    vector<vector<cv::Point>> contours;
    vector<cv::Vec4i> hierarchy;
    cv::findContours(blur, contours, hierarchy, cv::RETR_TRE
E, cv::CHAIN_APPROX_SIMPLE);

    // 6. Find largest contour
    int idx = 0;
    double max_area = 0;
    for (int i = 0; i < (int)contours.size(); i++) {
        double area = cv::contourArea(contours[i]);
        if (area > max_area) { max_area = area; idx = i; }
    }

    // 7. Draw largest contour on frame
    cv::drawContours(frame, contours, idx, cv::Scalar(0, 255,
255), 2);
    cv::imshow("Output", frame);

    if (cv::waitKey(10) == 'x') {
        cv::destroyAllWindows();
        video.release();
```

```
        break;
    }
}
```

---

16. Image Segmentation (GrabCut)

GrabCut with user-drawn rectangle

```
cv::Mat img = cv::imread("./assets/car.jpeg");
if (img.empty()) {
    cout << "Failed to load image" << endl;
    return -1;
}

// User selects ROI rectangle interactively
cv::Rect rect = cv::selectROI("Select Object", img, false, fa
lse);

cv::Mat mask(img.size(), CV_8UC1, cv::Scalar(cv::GC_BGD));
cv::Mat bgd_model, fgd_model;

// Run GrabCut (5 iterations)
cv::grabCut(img, mask, rect, bgd_model, fgd_model, 5, cv::GC_
INIT_WITH_RECT);

// Extract foreground: pixels labelled FGD or probable FGD
cv::Mat fg_mask = (mask == cv::GC_FGD) | (mask == cv::GC_PR_F
GD);

// Apply mask to get segmented result
cv::Mat result;
img.copyTo(result, fg_mask);

cv::imshow("Original", img);
```

```
cv::imshow("Segmented", result);
cv::waitKey(0);
cv::destroyAllWindows();
```

Full Program Examples (assembled from blocks)

Example A: Load image → Grayscale → Canny → Contours → Show

```
#include<bits/stdc++.h>
#include<opencv2/opencv.hpp>
using namespace std;

int main() {
    // [BLOCK] Load colour image
    cv::Mat img = cv::imread("./assets/picture.jpeg");
    if (img.empty()) { cout << "Failed to load\n"; return -1;
}

    // [BLOCK] BGR → Grayscale
    cv::Mat gray;
    cv::cvtColor(img, gray, cv::COLOR_BGR2GRAY);

    // [BLOCK] Canny edge detection
    cv::Mat edges;
    cv::Canny(gray, edges, 170, 250);

    // [BLOCK] Find contours
    vector<vector<cv::Point>> contours;
    vector<cv::Vec4i> hierarchy;
    cv::findContours(edges, contours, hierarchy, cv::RETR_EXT
ERNAL, cv::CHAIN_APPROX_NONE);

    // [BLOCK] Draw all contours
```

```
    for (size_t i = 0; i < contours.size(); i++)
        cv::drawContours(img, contours, i, cv::Scalar(0, 255,
0), 2);

    // [BLOCK] Display
    cv::imshow("Contours", img);
    cv::waitKey(0);
    cv::destroyAllWindows();
    return 0;
}
```

Example B: Webcam → Select ROI → Blob Detection Loop

```
#include<bits/stdc++.h>
#include<opencv2/opencv.hpp>
using namespace std;

// [BLOCK] Median helper
double median(vector<double> vec) {
    int size = vec.size();
    if (size == 0) return 0;
    sort(vec.begin(), vec.end());
    return (size % 2 == 0) ? (vec[size/2-1]+vec[size/2])/2.0
: vec[size/2];
}

// [BLOCK] Get median HSV
tuple<double,double,double> getMedianPixelValues(cv::Mat img)
{
    vector<double> h, s, v;
    for (int i = 0; i < img.rows; i++)
        for (int j = 0; j < img.cols; j++) {
            cv::Vec3b px = img.at<cv::Vec3b>(i, j);
            h.push_back(px[0]); s.push_back(px[1]); v.push_ba
ck(px[2]);
```

```
        }
    return make_tuple(median(h), median(s), median(v));
}

int main() {
    // [BLOCK] Open webcam
    cv::VideoCapture video(0);
    int hv = 0, sv = 0, vv = 0;

    // [BLOCK] ROI calibration loop
    while (video.isOpened()) {
        cv::Mat frame;
        video.read(frame);
        cv::namedWindow("Image", cv::WINDOW_NORMAL);
        cv::resizeWindow("Image", 640, 480);
        cv::imshow("Image", frame);
        if (cv::waitKey(10) == 'q') {
            cv::Rect bbox = cv::selectROI("Image", frame, fal
se, false);
            cv::Mat hsv; cv::cvtColor(frame, hsv, cv::COLOR_B
GR2HSV);
            cv::Mat obj = hsv(cv::Rect(bbox.x, bbox.y, bbox.w
idth, bbox.height));
            auto [h, s, v] = getMedianPixelValues(obj);
            hv = h; sv = s; vv = v;
            break;
        }
    }

    // [BLOCK] Blob detection loop
    while (video.isOpened()) {
        cv::Mat frame; video.read(frame);
        cv::Mat hsv; cv::cvtColor(frame, hsv, cv::COLOR_BGR2H
SV);

        cv::Scalar lower(hv-5, max(0,sv-50), max(0,vv-50));
```

```
        cv::Scalar upper(hv+5, min(sv+50,255), min(vv+50,25
5));
        cv::Mat masked; cv::inRange(hsv, lower, upper, maske
d);

        cv::Mat blur; cv::medianBlur(masked, blur, 5);

        cv::Mat blob_mask; cv::bitwise_and(frame, frame, blob
_mask, blur);
        cv::imshow("blob_mask", blob_mask);

        vector<vector<cv::Point>> contours;
        vector<cv::Vec4i> hierarchy;
        cv::findContours(blur, contours, hierarchy, cv::RETR_
TREE, cv::CHAIN_APPROX_SIMPLE);

        int idx = 0; double max_area = 0;
        for (int i = 0; i < (int)contours.size(); i++) {
            double area = cv::contourArea(contours[i]);
            if (area > max_area) { max_area = area; idx = i;
}
        }
        cv::drawContours(frame, contours, idx, cv::Scalar(0,2
55,255), 2);

        cv::namedWindow("Output", cv::WINDOW_NORMAL);
        cv::resizeWindow("Output", 640, 480);
        cv::imshow("Output", frame);
        if (cv::waitKey(10) == 'x') { cv::destroyAllWindows
(); video.release(); break; }
    }
    return 0;
}
```

Example C: Load image → HSV mask → Morphology → Contours → Show

```
#include<bits/stdc++.h>
#include<opencv2/opencv.hpp>
using namespace std;

int main() {
    // [BLOCK] Load
    cv::Mat img = cv::imread("./assets/picture.jpeg");
    if (img.empty()) { cout << "Failed\n"; return -1; }

    // [BLOCK] BGR → HSV
    cv::Mat hsv;
    cv::cvtColor(img, hsv, cv::COLOR_BGR2HSV);

    // [BLOCK] HSV mask (change bounds for your colour)
    cv::Mat mask;
    cv::inRange(hsv, cv::Scalar(36, 52, 80), cv::Scalar(86, 2
55, 255), mask); // green

    // [BLOCK] Median blur
    cv::Mat blur;
    cv::medianBlur(mask, blur, 5);

    // [BLOCK] Morphology — opening (remove small noise)
    cv::Mat kernel = cv::getStructuringElement(cv::MORPH_REC
T, cv::Size(3, 3));
    cv::Mat opened;
    cv::morphologyEx(blur, opened, cv::MORPH_OPEN, kernel);

    // [BLOCK] Find contours
    vector<vector<cv::Point>> contours;
    vector<cv::Vec4i> hierarchy;
    cv::findContours(opened, contours, hierarchy, cv::RETR_TR
EE, cv::CHAIN_APPROX_SIMPLE);
```

```
    // [BLOCK] Draw largest contour
    int idx = 0; double max_area = 0;
    for (int i = 0; i < (int)contours.size(); i++) {
        double area = cv::contourArea(contours[i]);
        if (area > max_area) { max_area = area; idx = i; }
    }
    cv::drawContours(img, contours, idx, cv::Scalar(0, 255,
0), 2);

    // [BLOCK] Display
    cv::imshow("Result", img);
    cv::waitKey(0);
    cv::destroyAllWindows();
    return 0;
}
```